CNN and GAN based Image De-Raining

Feby Elizabeth John
M.Tech Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore
febyelizabethjohn@gmail.com

M Prathyusha Lahari
M.Tech Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore
m.prathyushalahari@gmail.com

Chetan Nadiger
Valeo India Pvt. Ltd.
linkedin.com/in/chetanhn

***Abstract*— Rain-affected images pose significant challenges for computer vision systems across a wide range of industries, from surveillance and autonomous driving to remote sensing and photography. The presence of rain streaks degrades image quality, obstructs object visibility, and hinders reliable automated decision-making. To address this, we propose an image de-raining system that leverages deep learning techniques, specifically Convolutional Neural Networks (CNNs) and Conditional Generative Adversarial Networks (CGANs), to remove rain artifacts and restore clear, high-quality visuals.**

Our approach is trained and evaluated on a curated combination of benchmark datasets including Rain100H, Rain100L, Test100, Test1200, and Test2800, which consist of both natural and synthetically generated rainy images featuring varying rain densities. In addition to the benchmark data, we have introduced a novel synthetic rainy image dataset generated from drone-captured imagery, expanding the model's robustness to aerial perspectives — a feature especially valuable for applications like agriculture monitoring and disaster assessment.

The proposed models are assessed using two widely accepted image quality metrics: Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index Measure (SSIM). Our system is further enhanced with an intuitive, user-friendly web interface that enables users to upload rainy images, visualize side-by-side comparisons of input and de-rained output, and interpret performance metrics. Thus, our approach offers a scalable and reliable solution for rain streak removal, improving the effectiveness of systems operating under adverse weather conditions. We have also created synthetic rainy images of drone-captured images. We have applied synthetic rain augmentation to 100 outdoor drone-images to create a new variant suitable for image de-raining research.

Keywords*— *XAI, Quantum ML, CNN, CGAN, Image de-noising

I. Introduction

Rain is one of the most common weather phenomena on Earth, with approximately 50% of the planet's surface experiencing rainfall for over 100 days each year, according to the Global Rainfall Dataset. While rain is vital for ecological balance, its presence poses substantial challenges for vision-based systems. Studies have shown that the accuracy of traffic surveillance systems can drop by as much as 32% during rainy conditions, primarily due to visual obstruction and distortion caused by rain streaks. In the domain of autonomous vehicles, rain has been cited as a contributing factor in nearly 30% of sensor failures, according to a 2020 study by SAE International.

The process of rain streak removal, or image deraining, is a crucial pre-processing step designed to eliminate rain-induced visual artifacts from images, thereby improving clarity and interpretability. Clear images are particularly essential in safety-critical applications such as surveillance, autonomous driving, traffic monitoring, satellite imaging, and drone-based environmental assessment.

In this project, we present a deep learning-based image de-raining system using two architectures — CNN and CGAN. Our system is trained on a diverse dataset, combining standard benchmark collections — Rain100H, Rain100L, Test100, Test1200, and Test2800. We have generated a novel synthetic rainy image dataset from drone-based aerial images, which introduces new perspectives and rain densities into the training process. This hybrid dataset strategy enhances the model's generalization capabilities across various visual domains, from ground-level surveillance to aerial monitoring.

To evaluate the effectiveness of the proposed deraining models, we employ two quantitative metrics: Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index Measure (SSIM). These metrics provide objective insights into image restoration quality, allowing for standardized performance comparisons. Beyond model training and evaluation, we have also developed a dedicated user interface (UI) for this system, allowing users to upload images, select model architectures, view de-rained outputs alongside original inputs, and access real-time performance metrics.

Through this combination of deep learning techniques, synthetic dataset generation, and intuitive design, our system provides a comprehensive solution for rain streak removal, aiming to enhance the reliability and safety of vision-based systems under challenging weather conditions.

II. About the Application

2.1 Workflow of the Application

- Our product's UI interface begins with a Login Page.
- Once a User logs in, they come to the Landing Page, where the features of our product are highlighted. Also, the User can upload the rainy image they intend to de-rain and choose the model (CNN or IDCGAN) with which they want to do the de-raining process.
- Our product offers 3 features to Users:
  - De-rain an entire image
  - De-raining only a RoI: User can choose a Region of Interest on the uploaded image and our product will de-rain that specific region only.
  - Detection of Objects in the image in addition to de-raining

2.2 Module-wise details

Module name	Functionality	Forms
Login Module	Login Module for user to login to De-Rain Website	Login Page
Introduction Module	Webpage displays detailed introduction of De-Rain Project and Description about CNN, CGAN models. This page lists out the product's features. The User can also upload the image and choose the model the User wishes for the De-Raining process from a drop-down menu.	Introduction Page
De-raining Module	The selected model processes input image and removes rain streaks from it. In this page, the User can view the entire De-Rained image as output with the performance metrics shown.	De-Raining Page
Additional-Features Module	Additional Features were included to meet real-world scenarios, where user can select required region of interest in image or do object detection in given rainy image. the User can draw a RoI on the uploaded image, preview the selected RoI, and our product will de-rain that region alone as shown in Fig.4 and 5.	Region-of-Interest Page, Object Detection Page

Table 1: Module details of the UI Interface

2.3 UI Forms

The following figures are the UI interfaces of this proposed project.

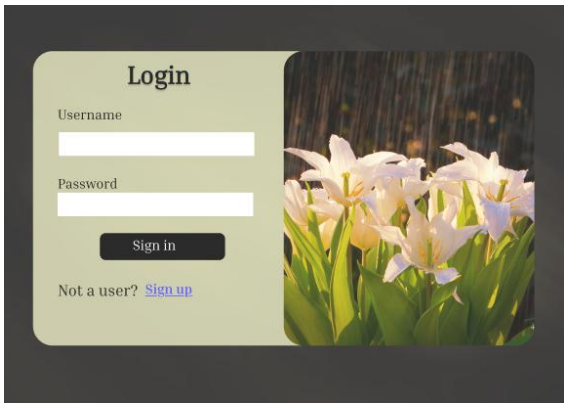

Fig 1: Login Page

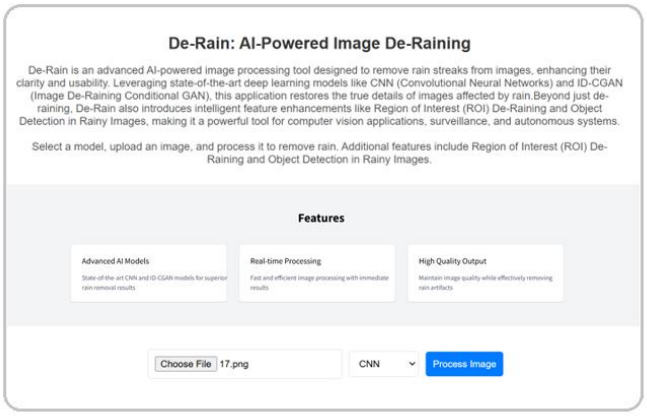

Fig 2: Introduction Page - Webpage displays detailed introduction of De-Rain Project and Description about CNN, CGAN models

Fig 3: De-Raining Page - In this module, the user can select model from drop-down menu, to choose the model they want to de-rain the image. The metrics of the de-rained image is displayed

2.4 Tech Stack (Frontend, Backend, UI)

- Front End: HTML, CSS3, Javascript
- Back end: Python
- Framework (Platform): Streamlit, Google Colab
- Image Processing Libraries: OpenCV, TensorFlow or PyTorch, Keras
- Language with Version: Python 3.x (latest stable version)

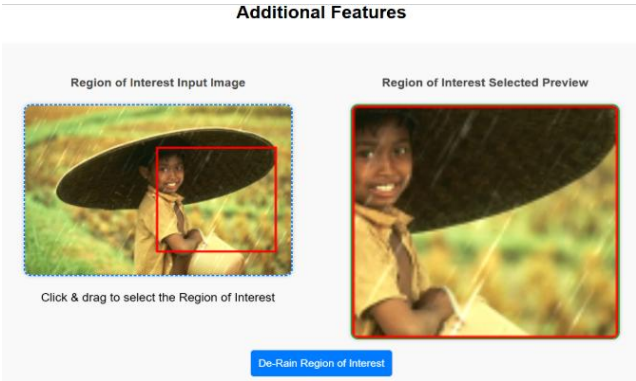

Fig 4: Region-of-Interest Page - User can select required Region of Interest in image to de-rain that region alone. U ser can see the preview of selected RoI and must click the "De-Rain Region of Interest" button

Fig 5: Output of de-rained Region of Interest

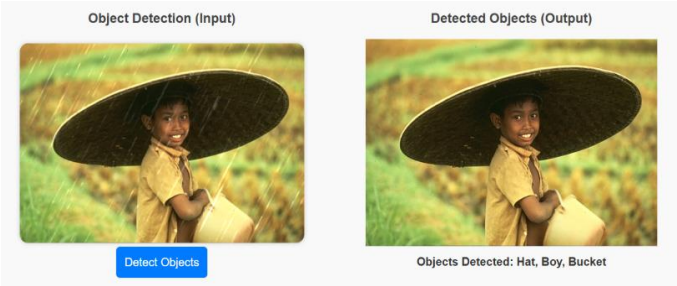

Fig 6: Object Detcion Page – This is an additional feature that enables detection of objects in a de-rained image

III. Dataset Description

A custom dataset of images split into training, validation, and testing datasets will be created from the Rain100H, Rain100L, Test100, Test1200, Test2800 datasets to create a mix of both natural and synthetic rainy images. The dataset will contain the rainy images and their corresponding ground truth images. This dataset contains images of different rain densities Light Rain, Heavy Rain Intensities. All these benchmark datasets contain augmented data with rotation and flipping.

In addition to these benchmark datasets, we have created a novel synthetic rainy image dataset from drone view images of outdoor scenes. 100 images were collected from the dataset used in Zhu et al. [1] for detection and tracking in drone-based vision tasks. The dataset aims to enhance the robustness of image de-raining by providing rainy image scenarios from a drone's point of view. No existing synthetic rainy image dataset covers drone images.

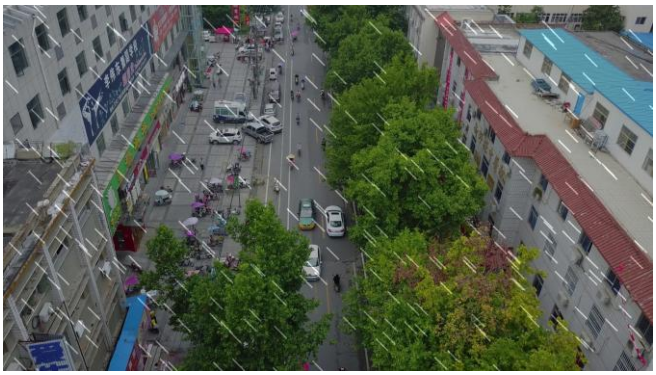

Fig 7: Generated synthetic rain on drone view images

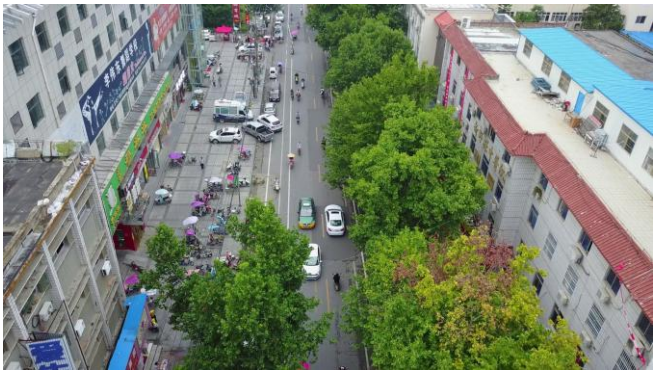

Fig 8: Ground Truth

IV. Related Work

Image de-raining has emerged as a critical research topic in the field of computer vision, particularly due to its significance in safety-critical systems such as autonomous driving, surveillance, and remote sensing. The rise of deep learning has significantly advanced de-raining methods, replacing traditional model-based techniques with powerful data-driven approaches.

Convolutional Neural Networks (CNNs) have proven particularly effective for image restoration tasks, including rain streak removal. Fu et al. [3] proposed an end-to-end deep learning architecture called Deep Detail Network (DDN), which separates high-frequency rain streak layers from background structures. The network successfully improves visual clarity by focusing on removing rain artifacts while preserving fine image details. Similarly, Wang et al. [4] introduced a Deep Shared Pyramid Network, which leverages a multi-scale feature extraction strategy to address varying rain densities and sizes, resulting in superior performance on both synthetic and real-world rainy datasets.

Yang and Lu [5] presented the Recurrent Hierarchy Enhancement Network (ReHEN), designed to progressively refine feature representations across multiple stages. This approach mimics the human visual system's iterative process for image perception and significantly boosts the network's ability to remove complex rain

patterns, even in dense rainfall scenarios. Later, Zamir et al. [6] proposed the Multi-Stage Progressive Restoration Network (MPRNet), a state-of-the-art framework that integrates stage-wise restoration, attention mechanisms, and feature fusion to handle severe weather-induced distortions.

Beyond CNNs, Generative Adversarial Networks (GANs) have been explored for their ability to generate visually convincing and perceptually high-quality de-rained images. Zhang et al. [7] introduced a Conditional GAN (cGAN) architecture specifically tailored for rain removal. Their approach combines a generator, which attempts to synthesize clean images, and a discriminator, which pushes the generator to produce more realistic de-rained outputs. Wang et al. [8] also employed conditional GANs and incorporated perceptual loss to enhance the output image's structural fidelity, further proving the effectiveness of adversarial training for deraining tasks.

In parallel to model development, the integration of explainable AI (XAI) tools has become increasingly relevant for enhancing transparency and trustworthiness in vision systems. Recent works, such as [9], have emphasized the role of explainability in interpreting neural network decisions, especially in the context of image restoration tasks where understanding the reasoning behind predictions is vital for real-world deployment.

Quantum-inspired methods have also been explored to optimize and accelerate image restoration pipelines. For example, the works discussed in [10–13] explore quantum signal processing techniques and their potential synergy with classical deep learning models, offering novel perspectives on computational efficiency and robustness in image processing.

Furthermore, the development of synthetic datasets has become an essential practice to overcome the limitations of real-world data scarcity. The use of datasets such as Rain100H, Rain100L, Test100, Test1200, and Test2800 has been foundational, while the generation of synthetic rain-augmented drone imagery, as conducted in this project, introduces a new domain-specific benchmark that helps models generalize across aerial perspectives and novel visual contexts.

V. METHODOLOGY

5.1 Architecture

5.1.1 CNN Architecture

A Convolutional Neural Network (CNN) is a Deep Learning algorithm which can take in an input image, assign importance (learnable weights and biases) to various aspects/objects in the image and be able to differentiate one from the other.

The Convolutional blocks (Conv Layer 1, 2, and 3 in Fig.7) serve as feature extraction blocks. They are three convolutional blocks with increasing filters (64 → 128 → 256 filters). Each block has BatchNorm and ReLU activation function.

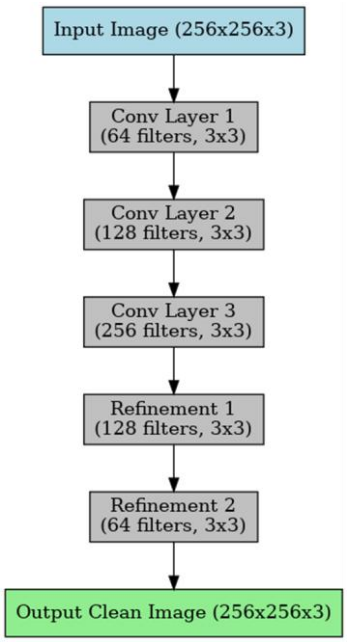

Fig 9: CNN Architecture Diagram

Convolution is a mathematical operation on two functions (f and g) that produces a third function that expresses how the shape of one is modified by the other.

Rain streak removal (de-raining) is done by Refinement layers 1 and 2 in the Fig.7. Finally, the Output Layer reconstructs the rain-free image with 3 filters (for RGB output) and Sigmoid activation.

The input image is passed through a CNN, which extracts rain streak features using multiple layers and filters. CNN learns to remove rain streaks while preserving important textures. After CNN processing, the output is a De-rained output image, where rain streaks have been significantly removed. Then, the PSNR, SSIM, and MSE are evaluated are displayed in the UI.

Before the image is processed by the CNN, we have added a low-pass filter to filter the base layer (smooth layer) and detail layer (rainy layer) [2] as shown in Fig.8.

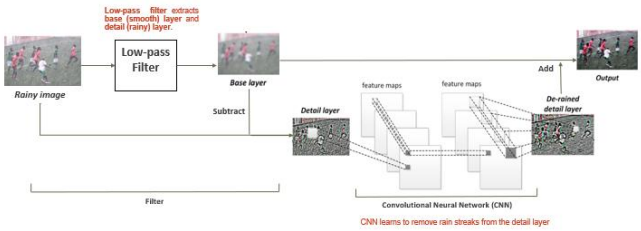

Fig 10: CNN-based image de-raining process using low-pass filter

In Fig.8, the process starts with a rainy image as input, containing both rain streaks and background details.

Layer decomposition splits the image into the sum of a "base" layer and a "detail" layer by using a low-pass filter for selective rain removal.

$$J = J_{base} + J_{detail}$$

The Base layer is extracted using a low-pass filter, which captures smooth, low-frequency background information.

$$\mathbf{x} = \mathbf{y} + \mathbf{w},$$

The Detail Layer is the result of subtracting the base layer from the original image, containing high-frequency details such as rain streaks. The detail layer is passed through a CNN which extracts rain streak features using multiple layers and filters. CNN learns to remove rain streaks while preserving important textures.

After CNN processing, the output is a De-rained Detail Layer, where rain streaks have been significantly removed.

For the final output reconstruction, the De-rained Detail Layer and the Base Layer are added back together to reconstruct a clean, de-rained image. This final output has rain removed while preserving original textures and details.

5.1.2 CGAN Architecture

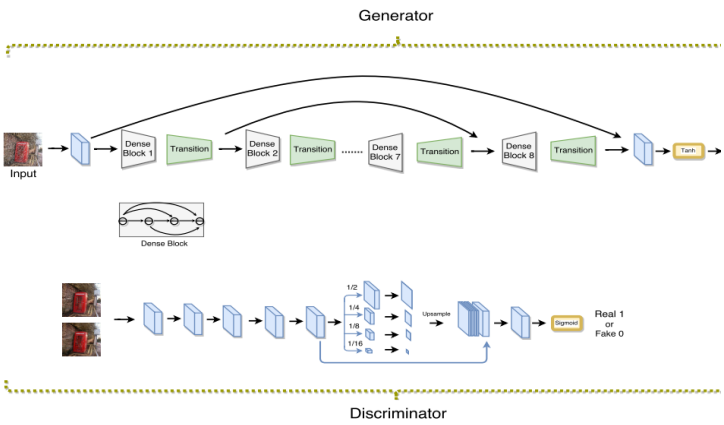

Fig 9: CGAN Architecture Diagram

The **Conditional Generative Adversarial Network (CGAN)** is a deep learning model designed to remove rain streaks from images while preserving the original content through identity consistency. It comprises two main components: a generator and a discriminator. The generator, built using a U-Net-like encoder-decoder architecture with skip connections, transforms rainy images into clean, de-rained outputs using convolutional layers, residual blocks, and Tanh activation. The discriminator, based on a PatchGAN structure, is a convolutional neural network that classifies image patches as real (clean) or fake (generated), using a Sigmoid activation for binary classification. Together, these networks ensure effective rain removal and image quality preservation.

Architecture
Input, num_c=3
3x3 Convolution, BN, ReLu, MaxP num_c=64
D(4 layers)+Td, num_c=128
D(6 layers)+Td, num_c=256
D(8 layers)+Tn, num_c=512
D(8 layers)+Tn, num_c=128
D(6 layers)+Tu, num_c=120
D(4 layers)+Tu, num_c=64
D(4 layers)+Tu, num_c=64
D(4 layers)+Tn, num_c=16
3x3 Conv, Tanh, num_c=3
Ourput, num_c=3

Fig 11: Network Architecture for Generator

Architecture
Input, num_c=6
3x3 Convolution, BN, ReLu, MaxP, num_c=64
3x3 Convolution, BN, ReLu, MaxP, num_c=256
3x3 Convolution, BN, ReLu, MaxP, num_c=512
3x3 Convolution, BN, ReLu, MaxP, num_c=64
Four-level Poling Module, num_c=72
Sigmoid
Output, num_c=72

Fig 12: Network Architecture for Discriminator

The CGAN model for image de-raining operates by taking a rainy image as input, which often contains obscured details due to rain streaks. The generator extracts essential features such as edges and textures through convolutional layers, and then uses residual blocks to learn and remove the rain components while preserving structural details. The decoder reconstructs the clean image with the help of skip connections to retain fine features. The discriminator then evaluates whether the output image is real (from the dataset) or fake (generated), using a PatchGAN approach to focus on local regions. Multiple loss functions, including generator and discriminator losses, guide the

training, resulting in a final rain-free image that closely resembles the clean ground truth.

5.2 Hyper-parameter table

Hyperparameter	CNN model Values
Epochs	10
Learning Rate	0.001
Optimizer	Adam
Batch Size	8
Early Stopping	Patience = 10
Loss functions	MSE, SSIM, PSNR

Table 2: Hyperparameter table for CNN Architecture

Hyperparameter	CGAN model Values
Epochs	50
Optimizer (Generator)	Adam (β_1 = 0.5, learning rate = 1e-4)
Optimizer(Discriminator)	SGD (Momentum=0.9, Decay=1e-6, Nesterov=False)
Loss Functions	Discriminator: Binary Cross Entropy (BCE) Generator: BCE + L1
Generator-Activation Function	Tanh
Discriminator-Activation Function	LeakyRelu, Relu
Discriminator-Output Shape	PatchGAN (30 × 30 × 1)

Table 3: Hyperparameter table for CGAN Architecture

The proposed CNN model (Table 2) for image de-raining was trained using a set of carefully chosen hyperparameters to optimize performance. The model was trained for 10 epochs with a batch size of 8, using the Adam optimizer with a learning rate of 0.001, which allowed for efficient and stable convergence. To prevent overfitting and ensure generalization, early stopping was implemented with a patience value of 10, meaning training would halt if no improvement was seen in the validation loss for 10 consecutive epochs. The training objective combined multiple loss functions, including Mean Squared Error (MSE) for pixel-wise accuracy, Peak Signal-to-Noise Ratio (PSNR) for image quality, and Structural Similarity Index Measure (SSIM) to capture perceptual similarity between the de-rained and ground truth images. These hyperparameters collectively supported robust learning while balancing model complexity and performance.

5.3 How is the proposed CNN model a light-weight and low-cost model?

The proposed CNN model is a lightweight and low-cost solution ideal for image de-raining, particularly in resource-constrained environments. With a training-validation-test split of 70:15:15, it maintains a balanced evaluation during training. The model includes Early Stopping to halt training once validation loss stops improving, effectively saving both time and computational resources. It has a small memory footprint of only 2.84 MB, containing approximately 743,939 parameters, each stored as a 4-byte float (float32). This is significantly smaller than typical deep models like ResNet-50 (~25M parameters) or VGG-16 (~138M), making it suitable for deployment on edge devices such as mobile phones and embedded systems. The model's efficiency is further supported by a short training time (just 10 epochs) and a small batch size of 8, which helps manage memory usage effectively. The architecture is intentionally kept simple, using only 3×3 convolutional kernels, no residual connections, and a stride of 1 throughout, which preserves spatial detail while maintaining a compact model design. Collectively, these choices result in a computationally efficient model that retains effectiveness without the overhead of complex deep architectures.

5.4 How is the proposed CGAN model a light-weight and low-cost model?

The proposed Conditional GAN (CGAN) model for image de-raining demonstrates a light-weight and low-cost architecture through its efficient design and training setup. Utilizing a balanced mix of hyperparameters—such as 50 training epochs, Adam optimizer for the generator (β_1 = 0.5, learning rate = 1e-4), and SGD for the discriminator with momentum and decay—it ensures both stability and speed during training. The model employs a PatchGAN discriminator output shape of 30×30×1 and uses Tanh, LeakyReLU, and ReLU activations for efficient gradient flow. Despite achieving high-quality results, the model maintains a moderate total parameter count of approximately 24.4 million, resulting in a model size of just 107.85 MB. With an low inference time of **0.7329** seconds (inference time per image), it avoids the heavy computational and memory demands of larger models. The use of simple optimizers (Adam and SGD) and no reliance on pre-trained backbones further reduces training costs.
And, computational cost of only ~12.10 GFLOPs, the CGAN is well-suited for real-time and resource-constrained applications. The efficient use of resources, combined with a 70:15:15 train-validation-test split for balanced learning, highlights the model's practicality for deployment in low-power or edge environments.

5.5 Model Compression Techniques – Lightweight Model

In recent years, deep learning has revolutionized numerous fields, including computer vision, natural language processing, and speech recognition. However, the state-of-the-art models in these domains are typically large and

computationally expensive, involving millions or even billions of parameters. Such models are often unsuitable for deployment on devices with limited hardware resources like smartphones, embedded systems, or edge computing platforms. To overcome this limitation, model compression techniques have been developed to reduce the computational burden and memory footprint of neural networks while preserving, as much as possible, their original performance. Among the most prominent and widely used techniques in this area are Pruning, Quantization, and Knowledge Distillation. These techniques allow developers to maintain the high performance of large models while making them more efficient and scalable for real-world applications.

5.5.1 Pruning

Pruning is a model compression technique that involves removing parameters in a neural network that contribute little to the model's output. The central idea is that many weights in a trained model are either redundant or insignificant in terms of their impact on the network's predictions. These weights can be eliminated to reduce the overall complexity of the model. Pruning can be done in several ways, including unstructured pruning, which removes individual weights regardless of their location, and structured pruning, which eliminates entire units such as neurons, channels, or filters. Unstructured pruning can lead to high sparsity in the model, but it often requires specialized hardware or software to fully take advantage of the resulting sparse matrix. Structured pruning, on the other hand, leads to more regular and hardware-friendly compressed models, as it reduces the size of the network in a structured manner.

The pruning process usually involves training a network to convergence, identifying weights or structures with the least importance (often based on weight magnitude or sensitivity analysis), and removing them. After pruning, the model typically undergoes a fine-tuning phase, where it is retrained on the dataset to recover from any loss in accuracy caused by the parameter removal. In practice, pruning has been shown to reduce the size of deep models by factors of 10x or more, especially in over-parameterized networks, making it a highly effective method for model optimization.

5.5.2 Quantization

Quantization is a compression technique that reduces the number of bits used to represent model parameters and intermediate calculations. Most deep learning models are trained and deployed using 32-bit floating-point (FP32) representations, which are both memory-intensive and computationally expensive. Quantization reduces this precision to lower-bit formats such as 16-bit floating-point (FP16), 8-bit integers (INT8), or even 1-bit binary values in extreme cases. The primary benefit of quantization is a significant reduction in model size and inference latency, which is especially valuable when deploying models on low-power devices or when performing real-time inference.

There are two main types of quantization: post-training quantization and quantization-aware training (QAT). Post-training quantization is applied after the model has been fully trained, by simply converting the weights and activations to a lower-precision format. This method is fast and easy to apply but can result in a noticeable drop in accuracy for some models, particularly those that are sensitive to small changes in weight precision. Quantization-aware training, on the other hand, simulates the effects of quantization during the training process itself, allowing the model to learn to be robust to quantization noise. This typically results in better accuracy retention. Additionally, some hardware accelerators and inference engines are optimized specifically for low-precision arithmetic, making quantized models faster and more energy-efficient on such platforms.

5.5.3 Knowledge Distillation

Knowledge Distillation is a unique model compression technique in which a smaller and more efficient model, known as the student, learns to mimic a larger and more accurate model, known as the teacher. Instead of training the student model using the original hard class labels (such as one-hot vectors), the student is trained using the soft output probabilities produced by the teacher model. These soft targets contain much more information than hard labels, as they reflect not just the predicted class but also the model's confidence in each class, capturing inter-class similarities. For example, if an image of a cat is classified as 90% cat, 5% dog, and 5% fox, the soft output reflects the semantic similarity between these classes, which can guide the student model toward better generalization.

The idea behind knowledge distillation is that the teacher network encapsulates a vast amount of knowledge about the training data and task in its learned representations and output distributions. By learning from these richer outputs, the student model can achieve performance levels close to that of the teacher model despite having fewer parameters and lower complexity. This technique is especially valuable when there is a need to deploy lightweight models that are suitable for fast inference or deployment in low-resource environments, without losing much in terms of prediction accuracy. Knowledge distillation has been successfully applied in a variety of domains, including image classification, natural language understanding, and speech recognition, and is often used in conjunction with other compression techniques like quantization or pruning for even greater efficiency.

These model compression strategies—Pruning, Quantization, and Knowledge Distillation—are crucial tools for making modern deep learning models accessible and practical in environments where computation, memory, or power consumption is a limiting factor. By strategically reducing redundancy and optimizing computation, these techniques ensure that AI models can be scaled beyond data centers and into the hands of everyday users and devices.

LIST OF METRICS

6.1 Mean Squared Error (MSE)

MSE measures the average squared difference between the original and reconstructed images.

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(x_i - y_i)^2$$

where, x_i is the value of the i-th pixel in the original signal, y_i is the value of the i-th pixel in the reconstructed signal, and N is the total number of pixels.

6.2 Peak Signal-to-Noise Ratio (PSNR)

PSNR is a logarithmic measure of the quality of the reconstructed image compared to the original.

$$PSNR = 10 \cdot \log_{10}\left(\frac{L^2}{MSE}\right)$$

where, L is the maximum possible pixel value (e.g., 255 for 8-bit images) and MSE is the Mean Squared Error.

6.3 Structural Similarity Index (SSIM)

SSIM evaluates the perceived quality of the image by comparing structural information.

$$SSIM(x,y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

where, μ_x and μ_y are mean of x and y, respectively, σ_x^2 and σ_y^2 are variance of x and y, respectively, σ_{xy} is the covariance of x and y, $C_1=(K_1L)^2$ and $C_2=(K_2L)^2$ are small constants to stabilize the division (commonly $K_1=0.01$, $K_2=0.03$), and L is the dynamic range of the pixel values (e.g., 255 for 8-bit images).

6.4 Mean Opinion Score (MOS)

MOS measures subjective quality based on human ratings.

$$MOS = \frac{\sum_{i=1}^{N} R_i}{N}$$

where, R_i is the rating given by the ith subject (on a fixed scale, e.g., 1–5 or 1–10) and N is the total number of subjects providing ratings.

VI. TESTABILITY OF THE MODEL

Deep learning (DL) testing has evolved significantly, introducing testing methods tailored to the unique structure and behaviour of neural networks. Traditional software testing strategies have inspired new test adequacy criteria for DL, with neuron coverage being a central metric. This measures how many neurons are activated during test runs, assuming that broader activation implies a higher likelihood of detecting faulty behaviour. Complementing this, surprise adequacy gauges how unexpected an input is relative to the training data, helping ensure the test set includes novel but relevant examples.

Among testing strategies, Differential Testing (DT) stands out for evaluating DL models by comparing the outputs of different models on the same input. It flags inconsistencies without needing a ground truth. Techniques like DeepXplore employ DT in conjunction with neuron coverage to uncover classification errors, although challenges remain in identifying which model's output is correct. Further advances include differential combination testing, integrating transformations to diversify input data.

Metamorphic Testing (MT) bypasses the oracle problem in DL by relying on metamorphic relations—logical expectations about how output should change given specific input transformations. If a DL model's output violates these relations, it suggests a fault. MT has been applied in systems like DeepTest to evaluate autonomous vehicle models under altered environmental conditions, and custom MR-based evaluations on the MNIST dataset have proven its ability to assess robustness under image transformations like rotation, shearing, and zoom.

Mutation Testing (MuT), adapted from traditional software testing, evaluates DL models by generating mutants—slightly altered versions of models or data—and checking if the test suite can detect behavioural differences. The tool DeepMutation exemplifies this by using source- and model-level mutations. Interestingly, MuT also helps identify adversarial examples, leveraging the fact that such inputs tend to react more strongly to minor model mutations.

Combinatorial Testing (CT) applies interaction-based test coverage principles to DL, examining how neuron combinations across layers influence outcomes. Tools like DeepCT focus on fixed and variable strength CT to uncover hidden defects by analyzing neuron interaction patterns post-layer. CT offers a scalable way to probe runtime states in large DL architectures, supporting early-stage fault detection.

Adversarial Perturbation Testing (APT) explores the model’s robustness by generating inputs that intentionally deceive the network. These adversarial examples are indistinguishable from valid inputs to humans but cause misclassification with high confidence. Research shows that even minimal perturbations can drastically affect predictions, posing risks especially in safety-critical applications. APT helps test the model’s vulnerability and assess how resilient it is to adversarial manipulation.

Together, these methods form a comprehensive framework for testing deep learning systems, addressing their unique complexities with structured, often automated, techniques that go beyond traditional software testing paradigms.

VII. TYPES OF GRADIENTS

The various types of gradients are:

7.1 Weight Gradients

- These are the gradients of the loss function with respect to the model parameters (weights and biases).

- These gradients determine how much each weight (or bias) in the network contributes to the loss.
- Used by optimizers (SGD, Adam, etc.) to adjust weights and minimize the loss.
- Key Role: They are directly responsible for learning in neural networks.

7.2 Activation Gradients

- These are the gradients of the loss function with respect to the activations of neurons.
- These gradients determine how the loss changes concerning the activation values of neurons in hidden layers.
- They flow backward during backpropagation and help compute weight gradients.
- Key Role: They propagate the learning signal through the network.

VIII. WHAT HYPERPARAMETERS CAN HELP SOLVE VANISHING GRADIENT PROBLEM?

- Learning Rate: A very small learning rate can make weight updates insignificant, slowing learning. Use an appropriate learning rate. $\alpha=10^{-3}$ (for Adam, RMSprop) or 10^{-2} (for SGD)
- Activation function choice: Sigmoid and Tanh squash values between 0 and 1 (or -1 and 1), making gradients small. Instead, use non-saturating activations like ReLU, Leaky ReLU, GELU, etc.

IX. RESULTS AND DISCUSSION

9.1 CNN

The performance of the proposed CNN architecture for image de-raining was evaluated using three key metrics: Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), and Structural Similarity Index (SSIM). These metrics were analyzed across training, validation, and testing datasets over 10 epochs, and the outcomes were benchmarked against a baseline CNN model reported in literature.

The training and validation loss (MSE) (Fig 13) shows a consistent decrease over the epochs, with training loss reducing from 0.009 to 0.006 and validation loss dropping from 0.068 to 0.0053. This indicates that the model successfully learned the underlying data distribution and generalized well to unseen validation data without significant overfitting.

As shown in Fig 14, both training and validation PSNR improved steadily, with final values of 22.97 dB and 22.86 dB, respectively. This reflects enhanced reconstruction quality as the network was trained.

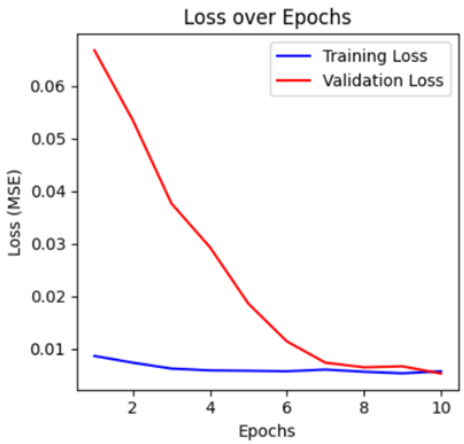

Fig 13: Plot of Training vs Validation Loss (MSE) over the epochs

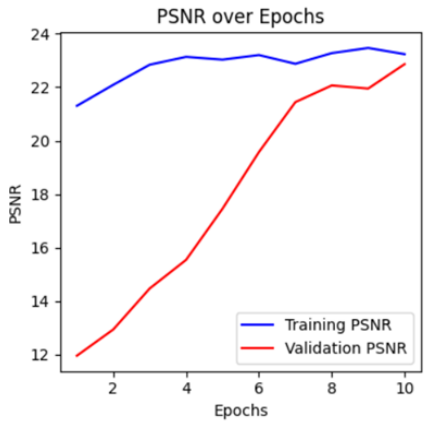

Fig 14: Plot of Training vs Validation PSNR over the epochs

Fig 15: Plot of Training vs Validation SSIM over the epochs

The SSIM scores (Fig 15 above) also followed a rising trend, showing improved structural fidelity of the reconstructed images. The final SSIM for validation surpassed that of training, reaching 0.7903, suggesting that the model was able to generalize well.

At the end of the 10th epoch, the model achieved the performance as shown in Table 4. This consistency across training and validation sets confirms that the model is neither overfitting nor underfitting, and is learning meaningful features for the image de-raining task.

	PSNR			SSIM		
	Training	**Validation**	**Testing**	**Training**	**Validation**	**Testing**
CNN	22.97	22.86	21.27	0.7679	0.7903	0.736

Table 4: Performance metrics of CNN Architecture

The proposed CNN achieved an Average PSNR of 21.27 dB, Average SSIM of 0.7368, and an Average MSE of 0.00894 on the test dataset. These values are reasonably close to the final validation scores, reinforcing the model's generalization capability to unseen data.

9.2 CGAN

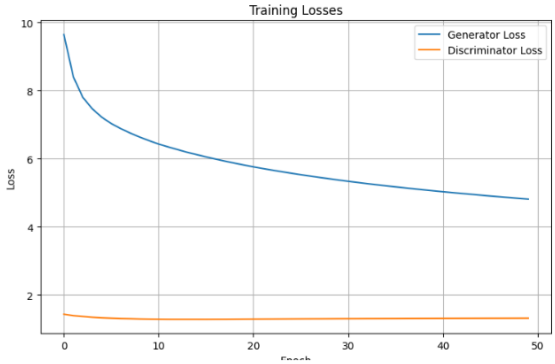

Fig 16: Training Loss plot of Generator and Discriminator over the training epochs

- A decreasing generator loss indicates that the generator is improving its ability to produce realistic images that can fool the discriminator.
- The discriminator loss remains low and stable, implying that the discriminator is performing well in distinguishing real from fake samples.
- The loss does not drop to zero — which is expected in GANs as in Fig. 16

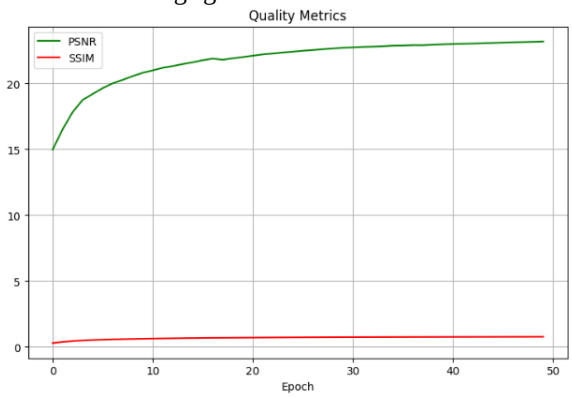

Fig 17: Plot of PSNR and SSIM over the training epochs CGAN

	PSNR		SSIM	
	Training	**Validation**	**Training**	**Validation**
CGAN	23.17	25.57	0.762	0.7970

Table 5: Performance metrics of CGAN Architecture

The model (Fig 17) is learning effectively. Sharp improvement in early epochs. Saturation at higher values indicates convergence. Both PSNR and SSIM show typical desired behaviour, Final SSIM value is around ~0.75 to 0.8 at epoch 50. SSIM value usually lies between 0 to 1. Generated images are progressively getting better in quality and structure similarity to ground truth

8.3 Comparison with other models

Results	PSNR	SSIM
Our CNN	22.97 (Training) 22.86 (Validation) 21.27 (Testing)	0.7679 (Training) 0.7903 (Validation) 0.736 (Testing)
Our CGAN	23.17 (Training) 25.57(Validation)	0.762 (Training) 0.7970 (Validation)
Baseline CNN	27	0.89 ± 0.06
Baseline CGAN	24.40	0.8275

Table 6: Comparison of Performance Metrics with that of Baseline Models

While the proposed CNN achieves good performance, the baseline model outperforms it significantly in both PSNR and SSIM. A PSNR gap of 5.7 dB and a SSIM gap of 0.15 indicates that while the current model is functional and learns meaningful features for de-raining, there remains substantial room for improvement.

This performance gap of proposed CNN with the baseline CNN may be attributed to differences in model architecture complexity (e.g., depth, use of residual connections or attention mechanisms), dataset diversity and size, training duration or optimization strategy, or data augmentation or preprocessing steps.

8.4 Final Output

8.4.1 CNN Final Output

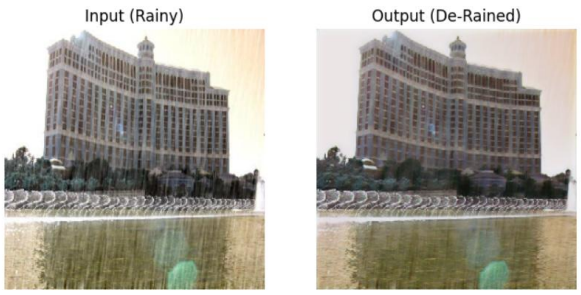

Fig 18: CNN Output

The Fig 18. shows the output of the proposed CNN when we tested with a rainy image. As can be seen, the CNN has effectively removed rain streaks in the output image.

8.4.2 CGAN Final Output

Fig 19: CGAN Output

The Fig 19. shows the output of the proposed CGAN when we tested with a rainy image. This Output image is obtained at after 50 epochs. As can be seen, the CGAN has effectively removed rain streaks in the output image.

X. CONCLUSION

The proposed CNN model shows consistent learning and generalization across datasets. Visual quality (as seen in PSNR and SSIM trends) of the CNN output improves significantly through training. However, compared to a high-performing baseline, the current architecture underperforms, highlighting potential areas for future improvement.

The CGAN model for image de-raining performed effectively, showing good reconstruction quality throughout training and validation. With solid PSNR and SSIM scores, and a final predicted image that visually looks clean and clear, the model successfully learns to remove rain streaks while preserving important image details. Overall, the CGAN proves to be a reliable and visually consistent solution for the image de-raining task.

XI. FUTURE WORK

This Project can be extended for real-time applications such as video stream processing in autonomous vehicles, drones, and surveillance systems. Optimization for edge devices like smartphones and IoT cameras would allow for low-latency, low-power consumption. Implementing Explainable AI (XAI) techniques like Grad-CAM and LIME would enhance model transparency and interpretability, crucial for high-stakes environments like autonomous driving. Additionally, these models could be optimized through methods such as pruning and quantization to improve performance and reduce deployment costs.

Retraining the models with diverse real-world data, along with incorporating user feedback, would further enhance its robustness. Ethical considerations, including bias reduction and data privacy, must also be addressed to ensure fairness and compliance with regulations. These steps would improve the model's generalizability and usability in real-world scenarios.

XII. REFERENCES

[1] Zhu, P., Wen, L., Du, D., Bian, X., Fan, H., Hu, Q., & Ling, H. (2021). Detection and tracking meet drones challenge. IEEE Transactions on Pattern Analysis and Machine Intelligence, 44(11), 7380–7399. https://doi.org/10.1109/TPAMI.2021.3076869

[2] X. Fu, J. Huang, X. Ding, Y. Liao, and J. Paisley, "Clearing the Skies: A Deep Network Architecture for Single-Image Rain Removal," IEEE Transactions on Image Processing, 2017

[3] X. Fu, J. Huang, X. Ding, Y. Liao, and J. Paisley, "Clearing the Skies: A Deep Network Architecture for Single-Image Rain Removal," *IEEE Transactions on Image Processing*, vol. 26, no. 6, pp. 2944–2956, 2017.

[4] C. Wang, X. Xing, G. Yao, and Z. Su, "Single Image Deraining via Deep Shared Pyramid Network," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 29, no. 11, pp. 3335-3349, 2019.

[5] Y. Yang and H. Lu, "Single Image Deraining via Recurrent Hierarchy Enhancement Network," in *Proceedings of the 27th ACM International Conference on Multimedia (ACMMM)*, Nice, France, 2019.

[6] S. W. Zamir, A. Arora, S. Khan, M. Hayat, F. S. Khan, M.-H. Yang, and L. Shao, "Multi-Stage Progressive Image Restoration," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14821–14831, 2021.

[7] H. Zhang, V. Sindagi, and V. M. Patel, "Image De-raining Using a Conditional Generative Adversarial Network," in *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 2020.

[8] Y. Wang, J. Liu, Z. Tang, C. Song, and Y. Chen, "Image De-raining Using Conditional Generative Adversarial Networks," in *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 2019.

[9] S. Ramachandran, N. Meena, and D. Ramachandran, "Explainable AI in Image Restoration: Interpretability for Denoising and De-Raining," *Neural Processing Letters*, 2023.

[10] M. A. Shukla, R. Dubey, and A. Saxena, "Quantum-Inspired Image De-Raining Using Signal Processing Models," *Signal Processing*, vol. 199, 2022.

[11] M. Abdelrahim et al., "Quantum Signal and Image Processing Using Symmetric Functions," *Symmetry*, vol. 15, no. 2, p. 396, 2023.

[12] S. Sharma, A. Joshi, and S. Yadav, "Quantum Mechanics Inspired Deep Learning for Image Restoration," *International Journal of Theoretical Physics*, vol. 59, pp. 3310–3328, 2020.

[13] M. Wang, L. Fang, and H. Zhao, "Quantum Image Processing: Theory and Applications," *Journal of Computer Science and Technology*, vol. 37, pp. 887–900, 2022.